Software Design

Module: CS5721

Lecturer: JJ Collins

Library Reservation System



Submitted by

Malleswari Kurva

25252704

Table of Contents

1. **Project Description**
   - **1.1 Overview**
   - **1.2 Business Description**
   - **1.3 Business Scenario**
   - **1.4 Case Tools**
2. **Software Lifecycle (Agile)**
   - **2.1 Lifecycle Adopted**
   - **2.2 Justification**
   - **2.3 Agile Practices Used in My Project**
3. **Project Plan**
   - **3.1 Validation of System Requirements – Functions**
   - **3.2 User Requirements (Students & Staff)**
   - **3.3 Librarian Requirements**
   - **3.4 System Requirements**
4. **Requirements**
   - **4.1 Functional Requirements**
     - **a) User Functional Requirements**
     - **b) Librarian Functional Requirements**
     - **c) System Functional Requirements**
   - **4.2 Use Case Diagrams**
     - **a) Search Books**
     - **b) Reserve Book**
     - **c) Borrow Book**
     - **d) Return Book**
     - **e) Manage Reservations**
     - **f) Auto Expire Reservation**
   - **4.3 Use Case Descriptions**
     - **Use Case 1: Reserve Book**
     - **Use Case 2: Borrow Book**
   - **4.4 Non-Functional Requirements**
5. **Architectural Patterns**
   - **5.1 Introduction**
   - **5.2 System Architecture**
   - **5.3 Analysis Class Diagram**
   - **5.4 Entity Relationship Diagram**
   - **5.5 Communication Diagram**
   - **5.6 Sequence Diagram**

6. Recovered Blueprints
    6.1 Recovered Architectural Hierarchy / Package Diagram
    6.2 Design Class Diagram
    6.3 Design Time Interaction Diagram

7. Diagrams
    7.1 Reservation State Chart
    7.2 Book State Chart

8. Component & Deployment Diagrams
    8.1 Component Diagram
    8.2 Deployment Diagram

9. Design Patterns
    9.1 MVC Pattern
    9.2 Repository Pattern
    9.3 Singleton Pattern
    9.4 Factory Method Pattern
    9.5 Strategy Pattern
    9.6 Observer Pattern (Notifications)

10. Added Value
    10.1 Architectural Value Through MVC
    10.2 Added Value Through Repository Pattern
    10.3 Centralized Data Handling Through Singleton
    10.4 Improved Flexibility with Factory Method
    10.5 Enhanced Search Capabilities Through Strategy Pattern
    10.6 Real-Time User Engagement Through Observer Pattern
    10.7 Improved Workflow for Library Staff and Users
    10.8 Overall System-Level Added Value

11. Code Fragements

12. Critique

13. References

1. Project Description

1.1 Overview

The Library Reservation System is intended to simplify the daily routines of an academic library by automating the processes involved. Through the system, students and staff can perform book searches, title reservations, borrow and return transactions. It implements a dynamic catalogue of records by managing restrictions on the number of titles borrowed, monitoring ongoing reservations, and automatically deleting unclaimed reservations after 48 hours. Librarians can control the book inventory, confirm transactions of borrow and return, and control the order of reservations. The system improves operational efficiencies, minimizes workload, and balances access to library resources.

1.2 Business Description

The old school operations of the library use manual record-keeping systems, which weakens the efficiency of the library. The library is never sure of the real-time availability of resources or the book is reserved, leading to frustration. Reservations are frequently inconsistent, unchecked with no oversight of books that are borrowed. The library spends too much time on record-keeping systems that they lose focus on more important tasks. Clearly, there is a need for some systems that automate book reservation, borrowing limits, inventory updating, and reservation expirations. It need not be more complex than a simple interface.

1.3 Business Scenario

Everyone logged into the library system can search for books in the system. If a book shows as available in the catalog, it can be reserved for future pickup. After 48 hours, the book is marked as available again, and the reservation is voided if the patron has not picked it up. The system manages the different borrowing limits of the students and the staff.Patrons can either self-serve, and borrow or return books, or they can wait for librarian patron approval for the same actions. Librarians manage the system by adding or removing books from the catalog, as well as editing book entries. They oversee borrow and return approvals, they can bypass the system's enforced limits, and they watch the reservation system queues. The system keeps a log of all transactions for each user to facilitate real-time inventory and to assure accountability and transparency.

1.3 Case Tools

1.1.1 Diagram Creation

- Draw**.io**

2. Software Lifecycle (Agile)

What is SDLC?

The Software Development Life Cycle (SDLC) is a systematic procedure for planning, designing, developing, testing, and maintaining software systems. It is a defined and systematic way to transform project requirements into a complete, coherent solution. SDLC usually breaks the work into well-defined phases such as Requirement Analysis, Design, Development, Testing, and Maintenance; each one of them guarantees that every stage in the project will be completed clearly and purposefully.

SDLC helps the developers understand what needs to be built, how it should function, how it should be designed, and how the final system needs to be refined and supported. Even in cases where the project does not involve actual coding as is the case with me the SDLC provides an overall framework for organizing tasks, producing deliverables, such as UML diagrams and architectural models, and maintaining consistency in the work.

2.1 Lifecycle Adopted

Examining the Library Reservation System . Libraries use around the world. I chose to explore the Agile Software Development Lifecycle. Agile focuses on developing deliverables in small increments with constant feedback and adaptability to change. It emphasizes working product and collaboration over documentation. Due to the nature of the academic projects with evolving issues and deadlines, Agile is a good fit.

2.2 Justification

Agile was the most suitable choice and perfectly aligned with the nature of the project.

The project work in this course was divided into two submissions, one in Week 7 and one in Week 13. This was in sync with the agile methodology used in the course. This methodology helped me focus on drafts for various use cases, diagrams and architecture, completing them first and refining them later. Constant refining and improving was better supported in the agile method than the more rigid waterfall method. What would eventually be a more conceptual UML diagram and system architecture, along with design patterns and system granular analysis, was agile in its nature and course did Added structure and a better workflow than the more rigid waterfall method. Having to pivot my work to complete design patterns, state machines and component diagrams later. The structure of the agile method with its focus on early and often deliveries enabled me to complete the course easily.

2.3 Agile Practices Used in My Project

I applied Agile methodology to manage my design workflow effectively.

Sprint Planning focus: Every new sprint (which is weekly) started with very precise goals I wanted to achieve, such as the completion of use case diagrams, construction of class diagrams, or designing sequence diagrams.

Design Iterations focus: In each succeeding week of the sprint, I focused on deepening or refining the system models I had previously worked on by adding new layers, which included but were not limited to communication diagrams, state diagrams, and deployment views.

Self-Review Retrospective focus: I took the time to think through, at the end of every sprint, which diagrams or design components needed more focus and improvement.

Task Management focus: I tracked progress and improvement in diagrams via a GitHub Scrum Board and created tasks like "create activity diagram", "refine class diagram", and "integrate strategy pattern", to simulate an Agile backlog Tracking. These iterative and flexible practices allowed me to clarify the structure of the design system. These methods allowed me to greatly enhance the discipline of my architectural practices, my system design practices, and helped me in the maintenance of quality in the diagrams, in the final system I developed for the Library Reservation System.

3. Project Plan

Validation of system requirements – functions.

User Requirements (Students & Staff)

- Allows
- Enable users to perform book searches (title, author, category, and ISBN).
- Allow users to check if/when a book is available.
- Enable the reservation of available books.
- Reserved books will be held for a period of 48 hours.
- Reservations will automatically be removed from the system after 48 hours.
- Users will be able to borrow books if the books are within their borrowing limits.
- Allow users to return books that are borrowed.
- Allow users to see their reservations.
- Enable users to see their borrowing limit and the quantity of books they currently have.

Librarian Requirements

- Add new books to the catalog.
- Edit and/or update book details.
- Remove books from the catalog.
- Approve a request to borrow a book.
- Approve a request to return a book.

- Temporarily increase the maximum number of books that can be borrowed.
- Make changes to reservations (cancel, reorder queue, confirm for pickup).

System Requirements

- System will keep user activity logs
- The system has to modify inventory counts after every borrow or return.
- The system should conduct a background

4 - REQUIREMENTS

4.1 Functional Requirements

a) User Functional Requirements (Students & Staff)

FR1 – User Authentication

All users (Students/Staff) should be able to register, log in, and log out securely.

FR2 – Search Books

The system shall allow users to search for books using filters such as:

- Title
- Author
- Category
- ISBN

FR3 – View Book Details

The system shall display book information including:

- Title, author, category
- Availability status (Available / Reserved / Borrowed)
- Reservation queue length (if any)

FR4 – Reserve Book

The system shall allow users to reserve a book that is currently available or will soon be available.

FR5 – Enforce Reservation Expiry

The system shall hold a user's reservation for exactly 48 hours, after which it automatically expires if the user does not collect the book.

FR6 – View Reservation Status

The system shall allow users to view:

- Active reservations
- Reservation expiry time
- Queue position (if waitlisted)

FR7 – Borrow Book

The system shall allow users to borrow a book if:

- The user's borrowing limit is not exceeded
- The book is available or reserved for that user
- Librarian approval is granted

FR8 – Return Book

The system shall allow users to return borrowed books through librarian assistance.

FR9 – Borrowing Limit Enforcement

The system shall enforce different borrowing limits:

- Students: lower limit
- Staff: higher limit

If the user attempts to exceed the limit, borrowing/reservation shall be denied.

FR10 – Cancel Reservation

Users should be able to self-service cancel their active reservations.

b) Librarian Functional Requirements

FR11 – Add Book to Catalog

Librarians can add new books to the library catalogue.

FR12 – Update Book Information

Librarians can edit book attributes such as title, author, category.

FR13 – Remove Books

The system shall allow librarians to delete books from the catalogue.

FR14 – Approve Borrow Requests

The system shall allow librarians to approve or reject borrowing requests from users.

FR15 – Approve Return Requests

The system shall allow librarians to process and confirm the return of books.

FR16 – Manage Reservations

The system shall allow librarians to:

- View all reservations
- Cancel reservations
- Modify reservation order
- Validate reservation pickup

FR17 – Override Borrowing Limits

The system shall allow librarians to override user borrowing limits in special cases.

C) System Functional Requirements

FR18 – Automatic Reservation Expiry

System should be able to auto expire reservations after 48 hours as background scheduling or a timer.

FR19 – Inventory Update

The system shall automatically update book status after borrowing or returning:

- Available
- Reserved
- Borrowed

FR20 – Activity Logging

The system shall log every significant action such as:

- User logins
- Book reservations
- Book borrow/return events
- Catalog changes

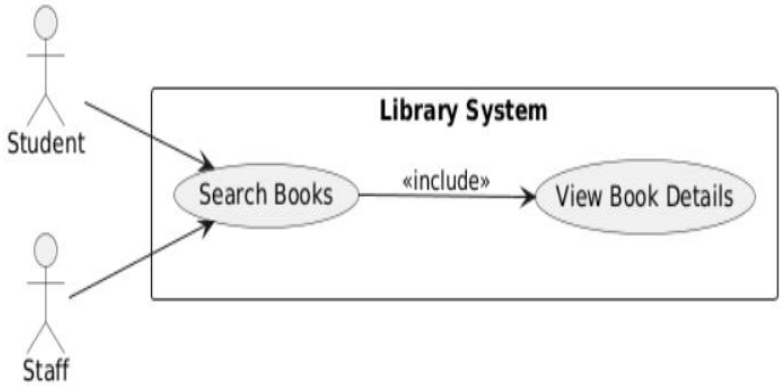
FR21 – Notification System

The system shall notify users in these cases:

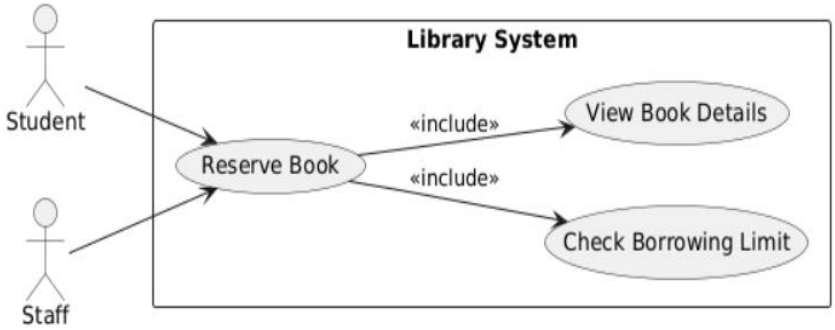
- Reservation created
- Reservation expiring soon
- Reservation expired
- Book available for pickup

4.2 Use Case Diagrams

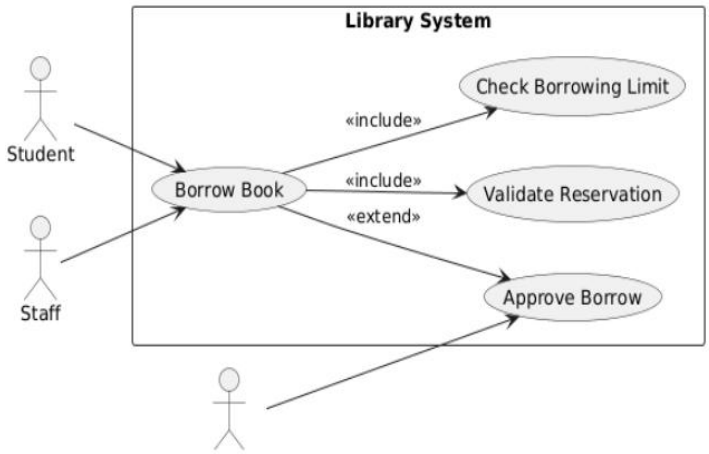
a) Search Books



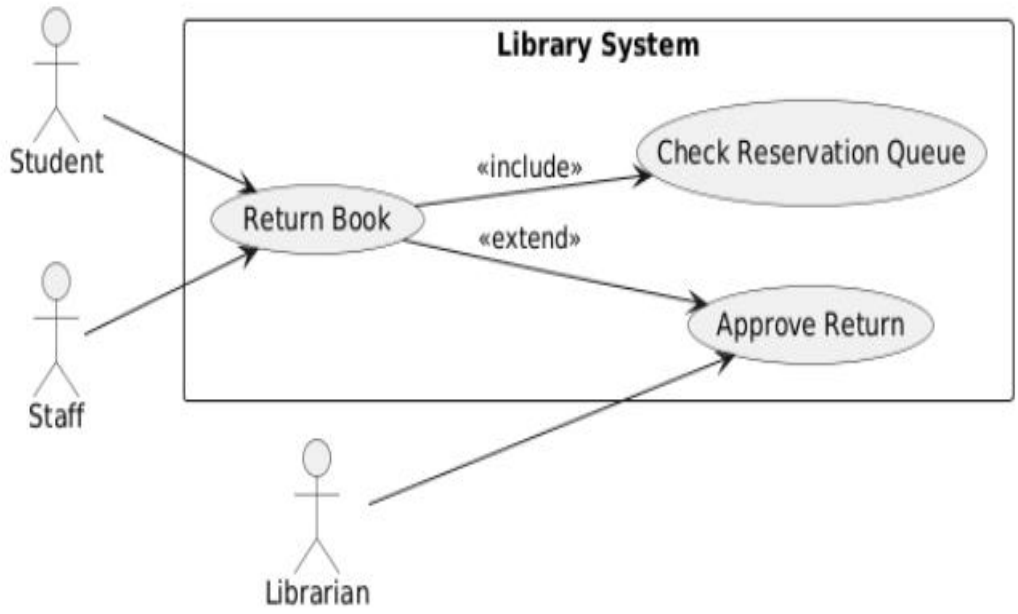
B) Reserve Book



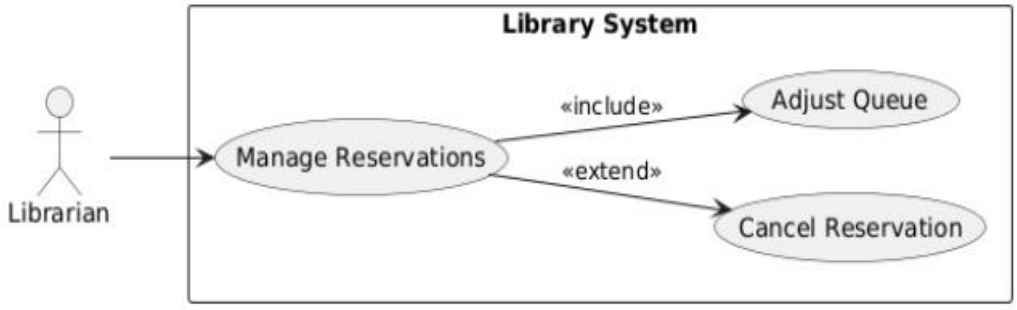
C) Borrow Book



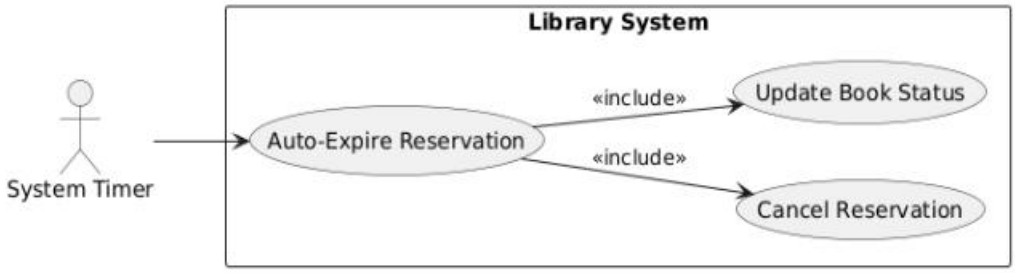
D) Return Book



E) Manage Reservations



F) Auto Expire Reservation



4.3 Use Case Descriptions

Use Case 1: Reserve Book

Use Case Name	Reserve Book
Actor(s)	User (Staff or Student)
Preconditions	User is authenticated. Book exists in catalog.
Postconditions	Reservation is created with status PENDING. Book is on hold for 48 hours.
Main Success Scenario	1. User performs the search on the book. 2. System shows book with its availability. 3. User clicks on book and selects Reserve. 4. System validates the reservation limit of the user. 5. System records the reservation and modifies the state to PENDING. 6. System activates the timer for the expiration (48 hours). 7. The record is sent to the user for acknowledgment.
Extensions (Alternate Flows)	4a Alternate Access User has exceeded their allowed limit on reservations. System shows the error. 3a Capture Book is currently on hold for others. System indicates Unavailable.

Quality attributes	Efficiency of system in conducting the lookup and reserving a book. Extensibility. The system is designed to allow addition of different resource types such as journals
Special Requirements	System should automatically expire the reservation in 48 hours. System should track the reservations. The reservations should seamlessly change states

Use Case 2: Borrow Book

Element	Description
Use Case Name	Borrow Book
Actor(s)	User (Staff or Student)
Preconditions	Book available for pickup. - User has an active reservation.
Postconditions	Book status indicates that it has been loaned- Reservation status changes to FULFILLED..
Main Success Scenario	1. User enters library. 2. Librarian confirms reservation. 3. System checks reservation status. 4. System changes book status to Borrowed. 5. System changes reservation status to FULFILLED 6. User receives receipt
Extensions (alternate flows)	3a. Reservation has expired. User is unable to borrow. 2a. Book is not present. Librarian documents problem.
Quality Attributes	Flexibility: Future incorporation of self-service checkout. Trustworthiness: Reliable tracking of which items are currently loaned.
Special Requirements	Reservation has to be fulfilled before its expiration. Different user types are subject to various borrowing quotas.

4.3 Non-Functional Requirements

1. Extensibility

The system should have the ability to easily add desired features, even if that means having to make changes to the system.

Project Implementation:

- Utilization of the **Model View Controller** paradigm guarantees diversity of responsibilities.
- Implementation of the **Repository design pattern** decouples data persistence, enabling an eventual adoption of a database as opposed to files systems.
- Other design patterns deployed, such as Factory Method and Observer, allow for straightforward integration of new user roles (Alumni, for example) or new rules pertaining to booking.

2. Performance

To perform the way we expect, the system should be designed such that the system should respond to the user actions promptly, regardless of the size of the dataset.

Project Implementation:

- Utilization of a HashMap to access books by value enables minimal search time, since lookups in HashMap collections are done in memory.
- The overheads of File I/O are minimized by adding a batch to the I/O.
- The processing in the system is minimized by having a lightweight console used as the interface.

3. Reliability

The system is required to reliably apply the rules of the business and keep the data intact.

Project Implementation:

- There is the automated JUnit testing of a few major use cases.
- There are rules in place with regards to booking and it is not possible to double-book the same book.
- The system is designed to handle errors so that it does not crash with invalid input.

4. Usability

The system should not be difficult to use for either students or staff.

Project Implementation:

- The use of straightforward console-based menus where users are taken through the steps sequentially with instructions.

- Users are given clear instructions and feedback messages (e.g. “You have successfully reserved the book” or “This book has already been reserved”).

5. Maintainability

The ability to easily maintain and update a system.

Project Implementation:

- Modular package structure. (model, service, repository, ui)
- Coding style and comments made clear and consistent.
- Version control in GitHub allows to track changes and collaborate on development.

5. Architectural Patterns

Introduction

These Days Libraries must keep track of the number of books in their collection, the number of users and their accounts, and all of the user reservations, they need to have some kind of system to do all of that efficiently, and the **Model-View-Controller (MVC)** architectural style can help to do that efficiently.

In the MVC approach,

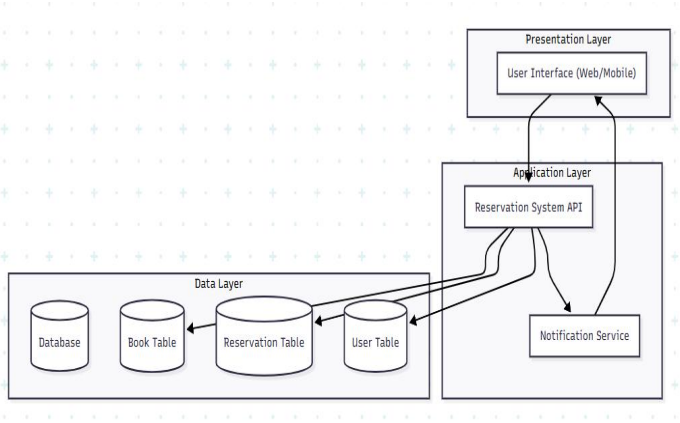
- The **Model** refers to all the **data and the business logic**, which in this case are the books, users, and reservations, along with business rules like loan rules, overdue fines, etc.
- The **View** refers to the part of the system that contains the **user interfac**e and displays information in various formats, like search pages, reservation or loan confirmations, and dashboards/overviews.
- The **Controller** refers to the part of the system that **receives user input and requests**, validates the requests, modifies the model, and renders the view/returns the response. This approach/architecture is a good fit for a library reservation system because it enables separation of concerns. This means the system will be easier to maintain, and adding features related to reservations of e-books or other future library resources will be much easier.

The system can also be reused on different platforms like on the web and mobile applications. Each individual part of the system can be tested on its own. In actual practice, MVC is implemented by creating entities and rules in the model, then user interface views in the system are designed, and user actions are managed in the controller, which allows for a consistent workflow in which users are be able to search for books, reserve them, and cancel reservations, while the system updates and displays information in real time.

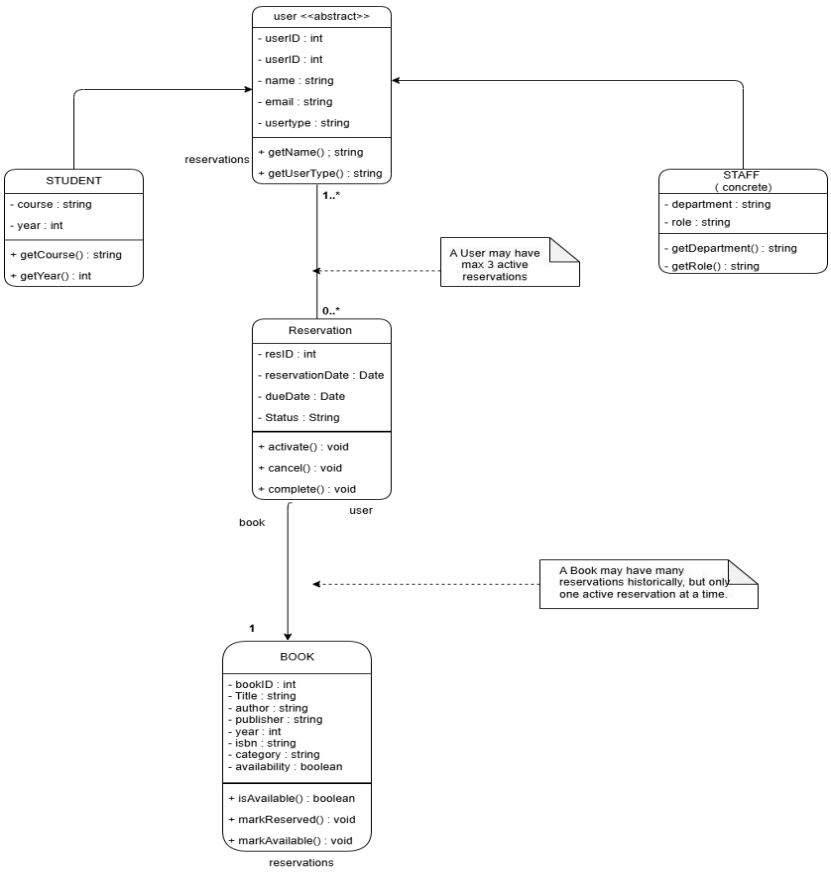
System Architecture

Overview

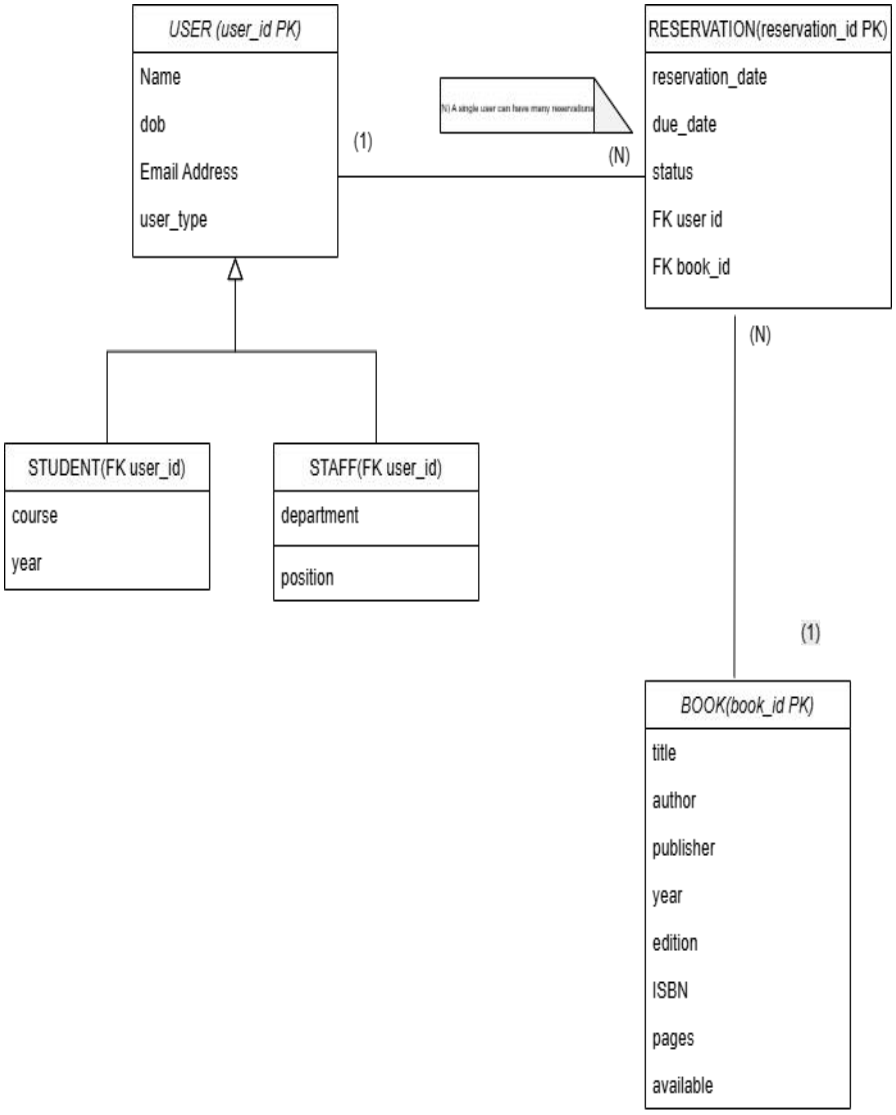
The Library Reservation System is designed using the Model-View-Controller (MVC) architectural approach, along with the Repository pattern for the management of data persistence. This gives the architecture a great separation of concerns as well as modularity and extensibility. This architecture is simple and prototype-appropriate, but still scalable for future developments.



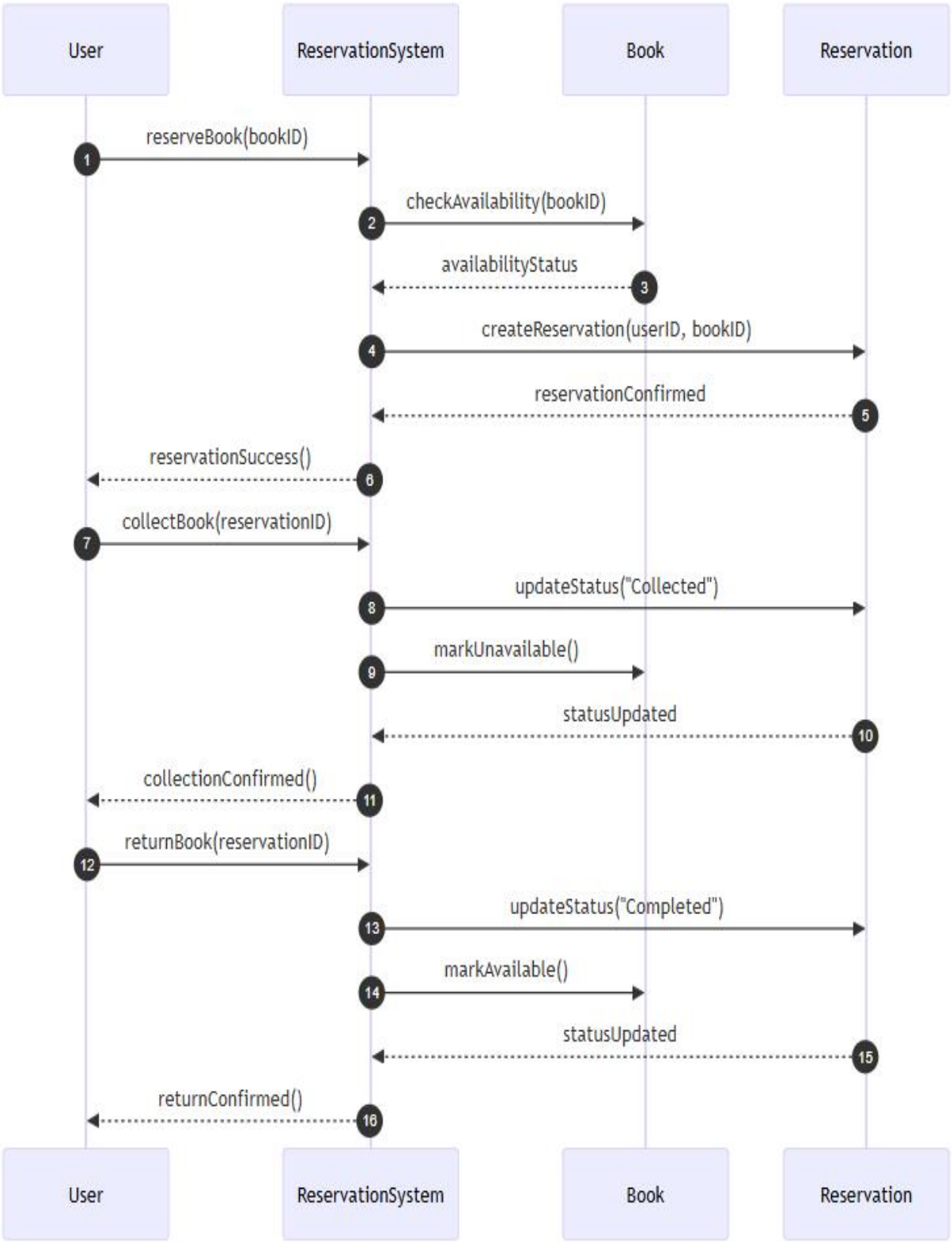
5.1) Analysis class diagram



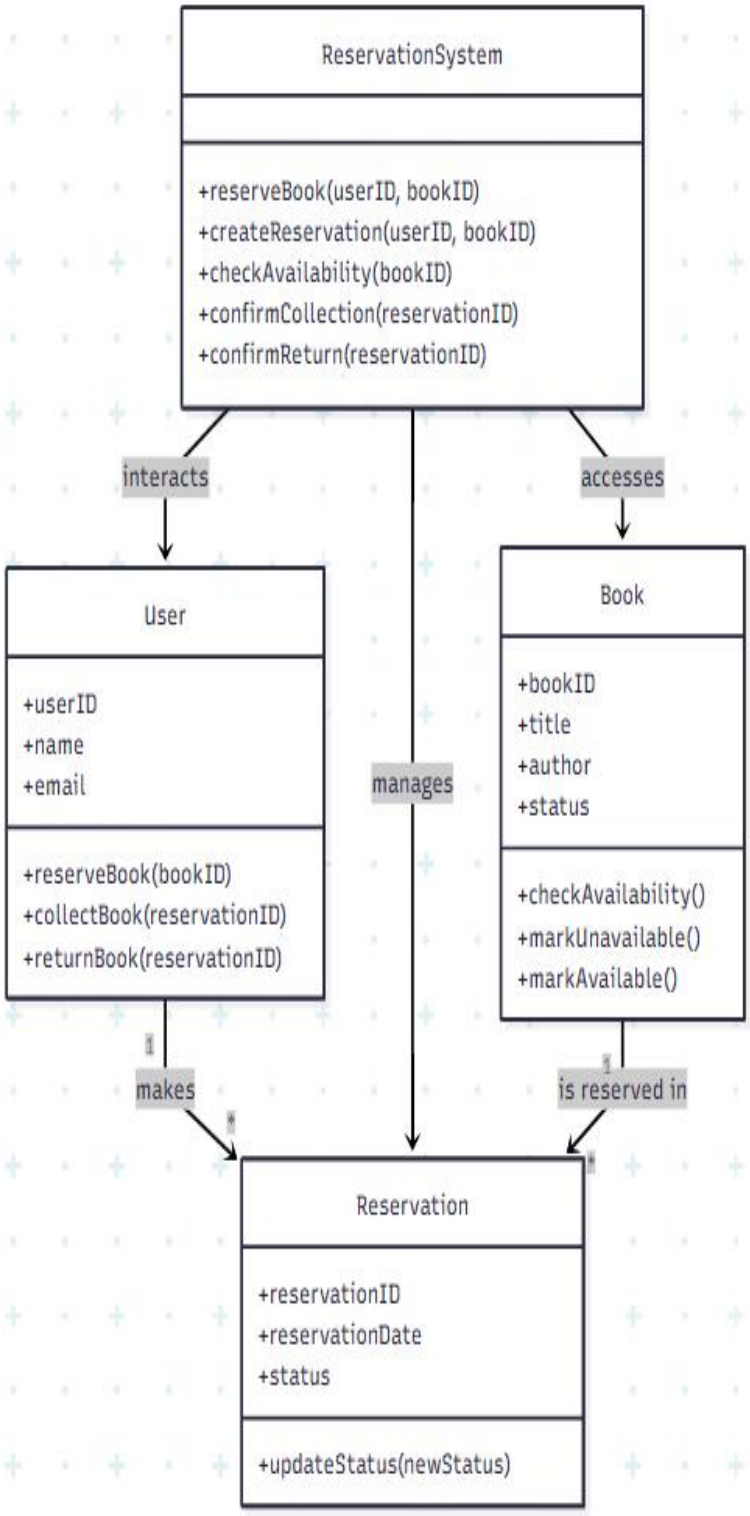
5.2) Entity Relationship Diagram



5.3) Communication Diagram

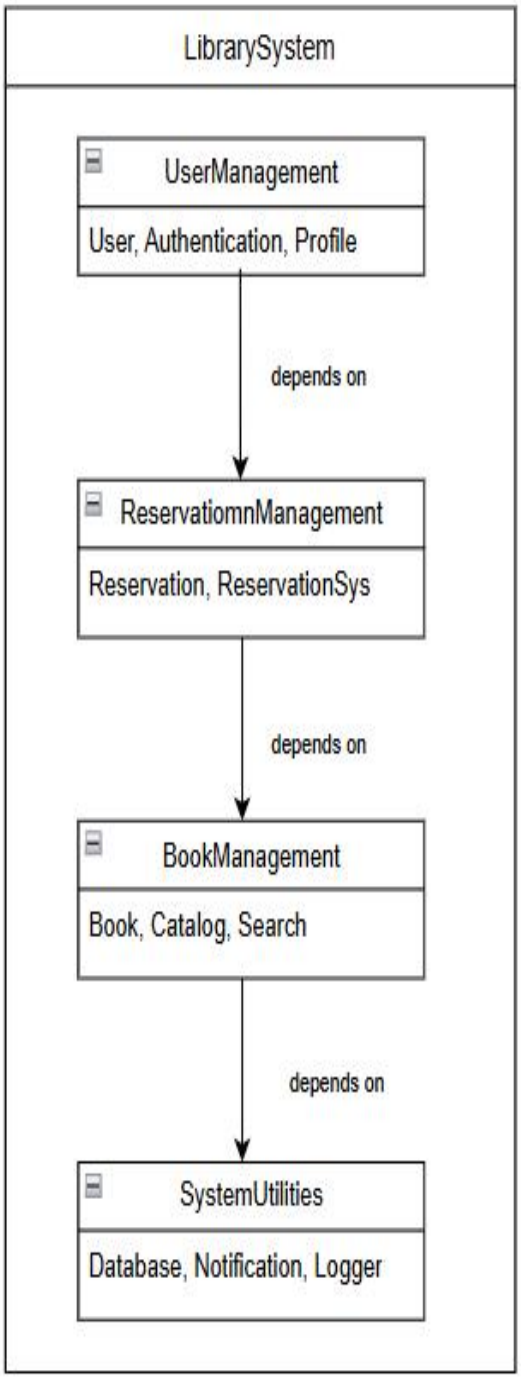


5.4) Sequence Diagram



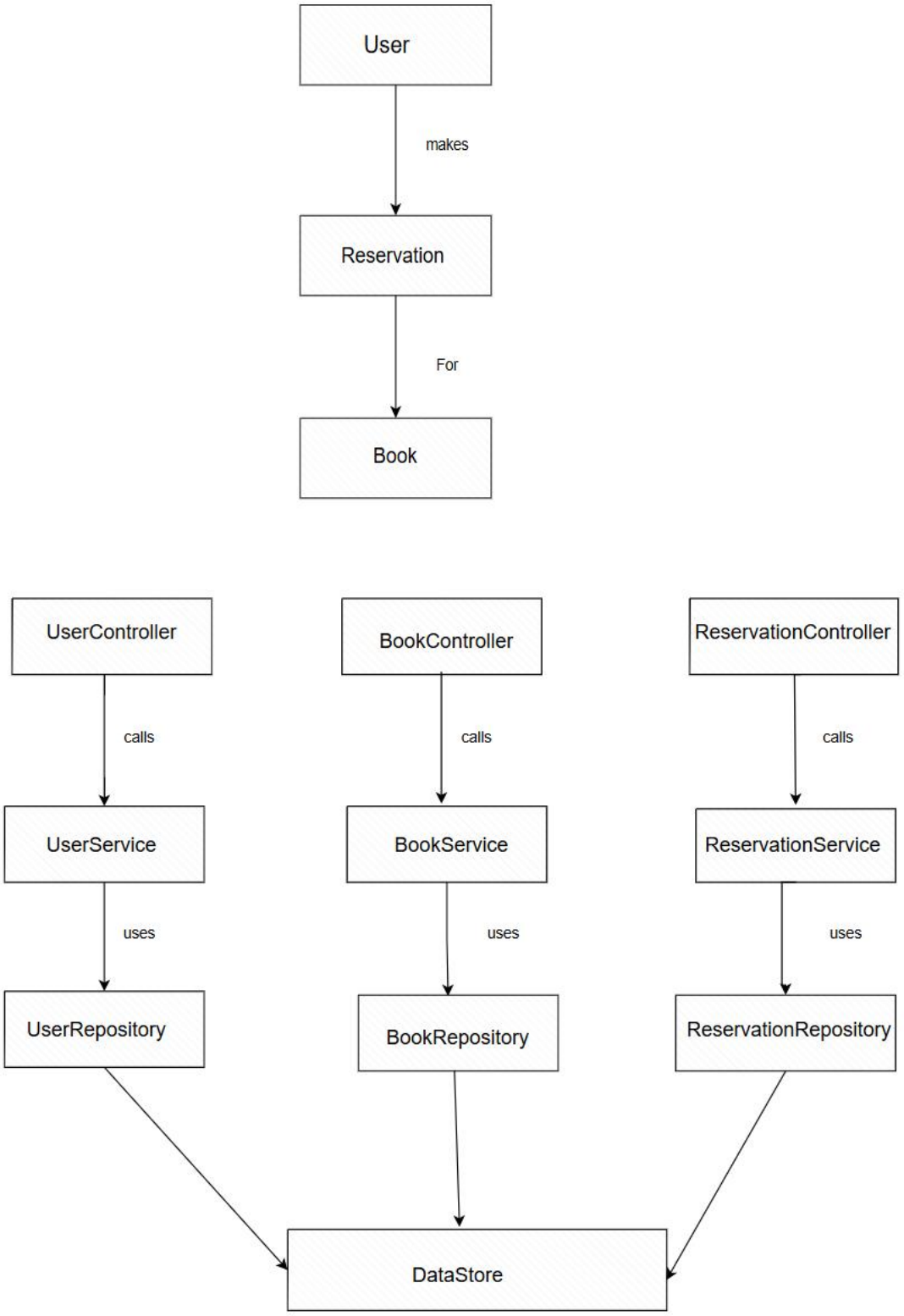
6. Recovered Blueprints

6.1 Recovered Architectural Hierarchy/ Package Diagram

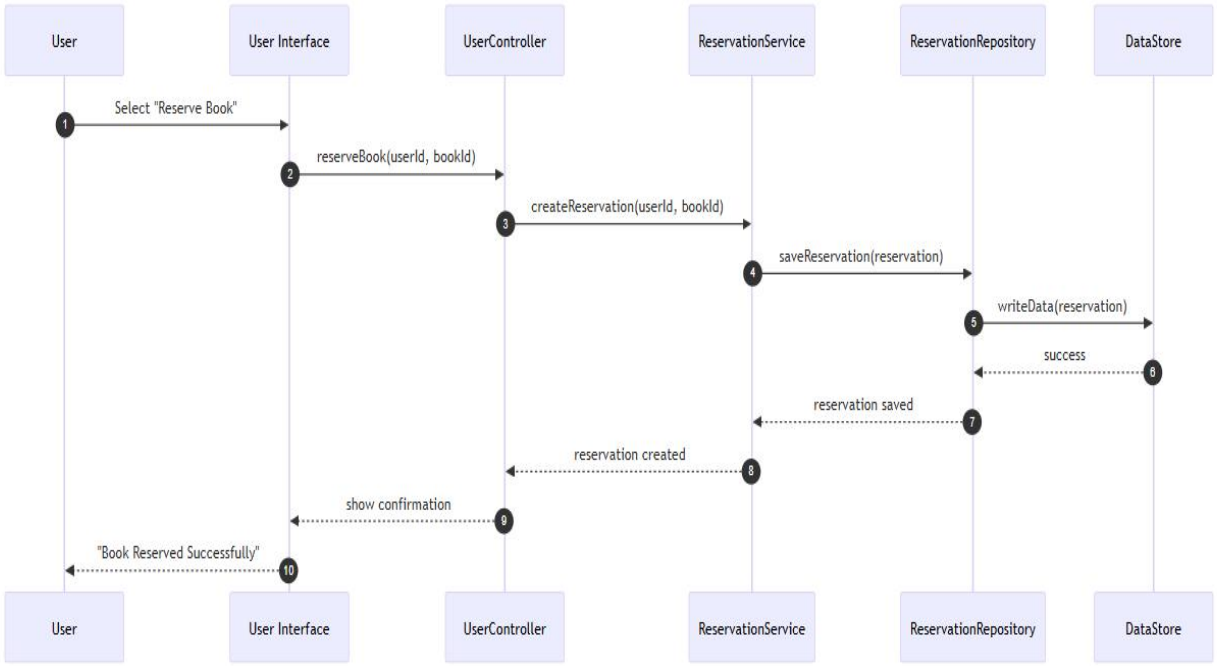


Dependencies:
- UserManagement → ReservationManagement
- ReservationManagement → BookManagement
- All packages → SystemUtilities

6.2 Design Class Diagram

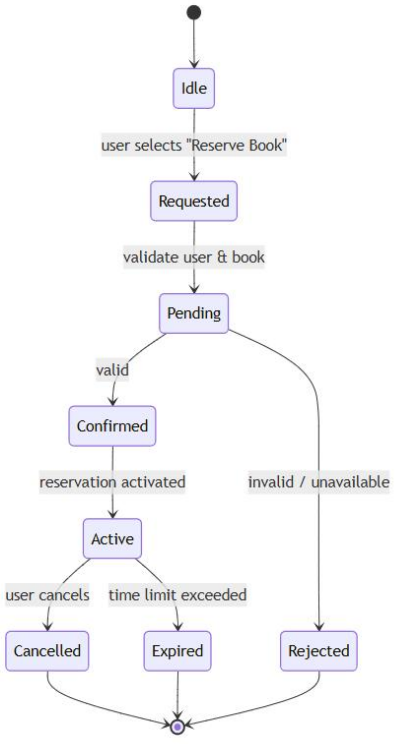


6.2 Design Time Interaction Diagram

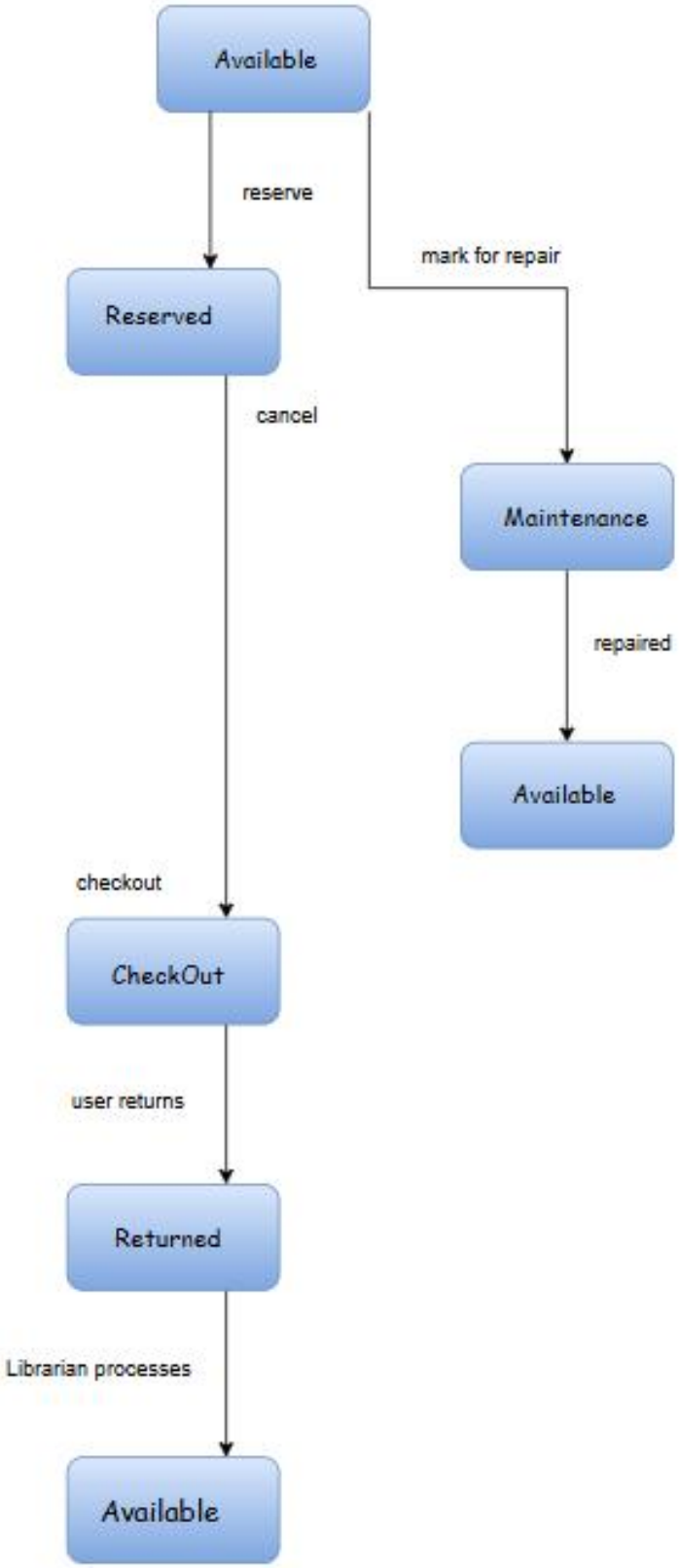


7. Diagram

1. Reservation State Chart

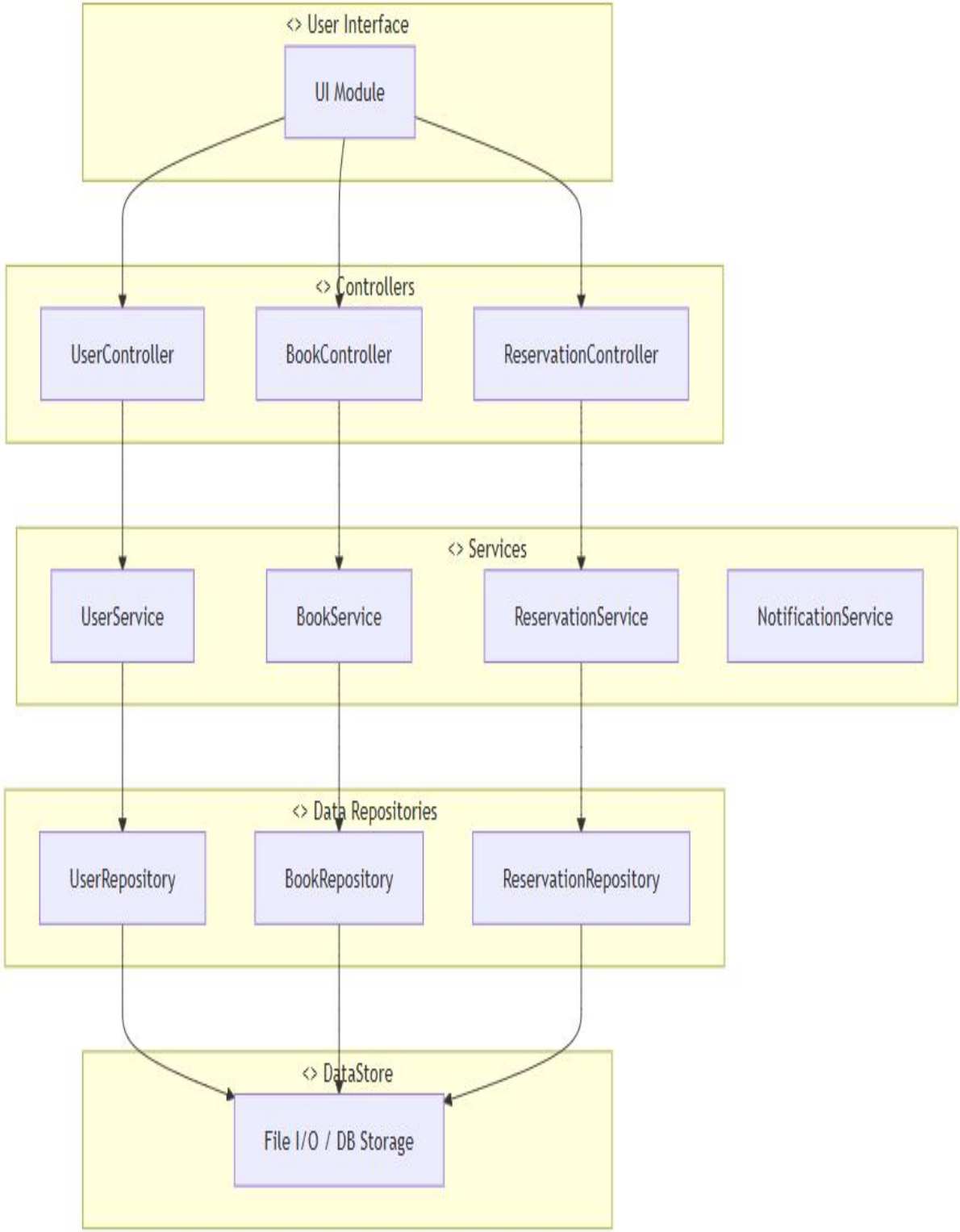


2. Book State Chart

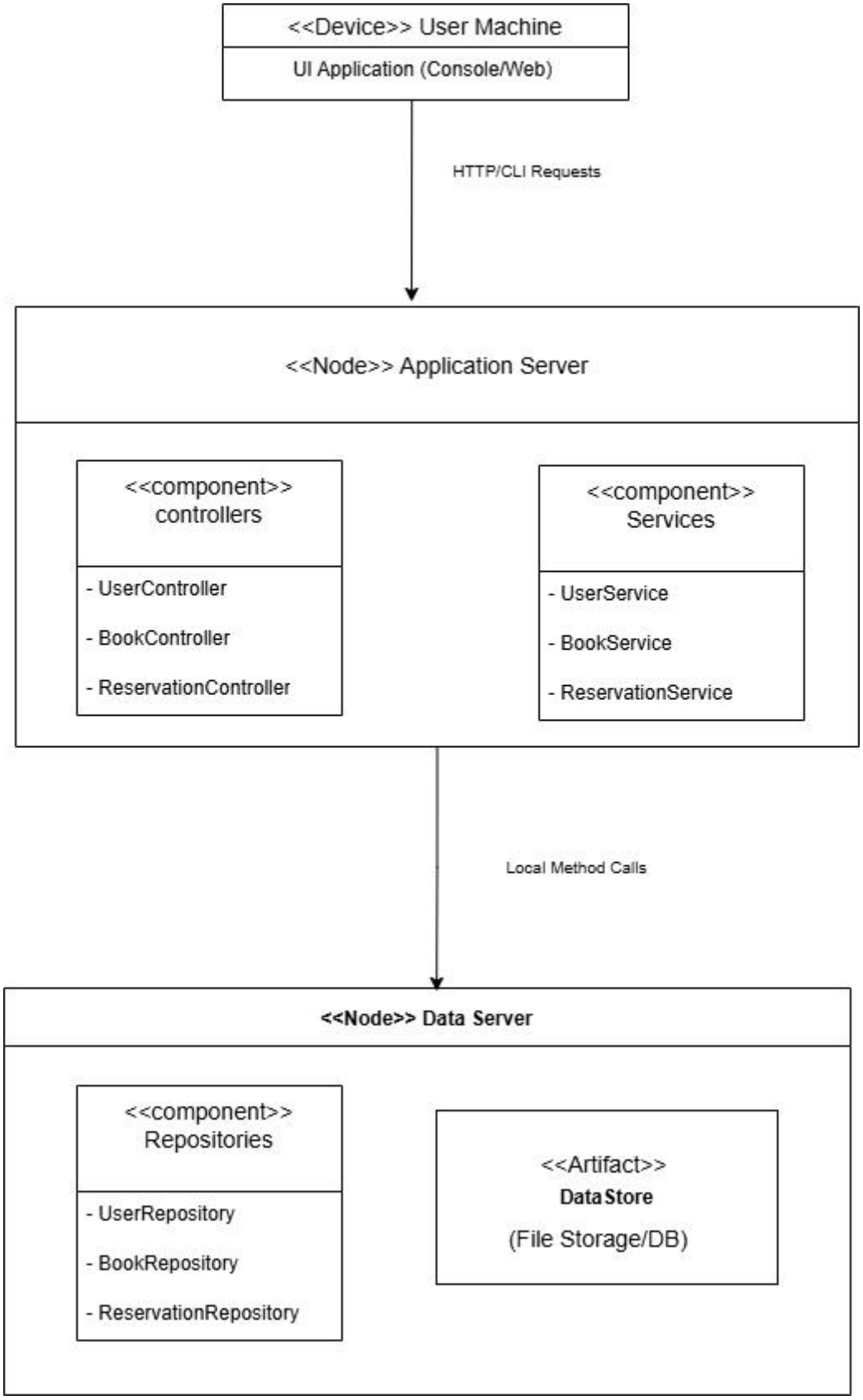


8. Component & Deployment Diagrams

8.1 component diagram



8.2 Deployment Diagram



9. Design Patterns

9.1 MVC Pattern

Definition

The Model–View–Controller (MVC) pattern is an architectural pattern which divides an application into 3 parts, which are interconnected on multiple sides.
The parts are:

Model : which represents the data and business rules
View : It has the user interface and Business rules
Controller: which handles user actions and interaction and coordination between Model and View. The separation improves modularity, and allows the different components (UI, logic and data) to be adjusted independently.

Implementation

The Library Reservation System implements the Model View Controller (MVC) architectural pattern to ensure separation of concerns. The Model layer is made of domain entities such as User, Book, and Reservation which encapsulate and focus on the core data structures and their rules. The View layer is realized by the interface to the system which captures user input and presents system outputs. The Controller layer which is realized by User Controller, Book Controller and Reservation Controller is responsible for fetching the actions and updating the view by performing the appropriate business logic.By splitting the system into the three layers, the application is made maintainable, testable and extensible. User interface changes do not implicate business logic and the model structures changes do not compromise UI components. This leads to improved modularity and scalability within the system design.



9.2 Repository Pattern

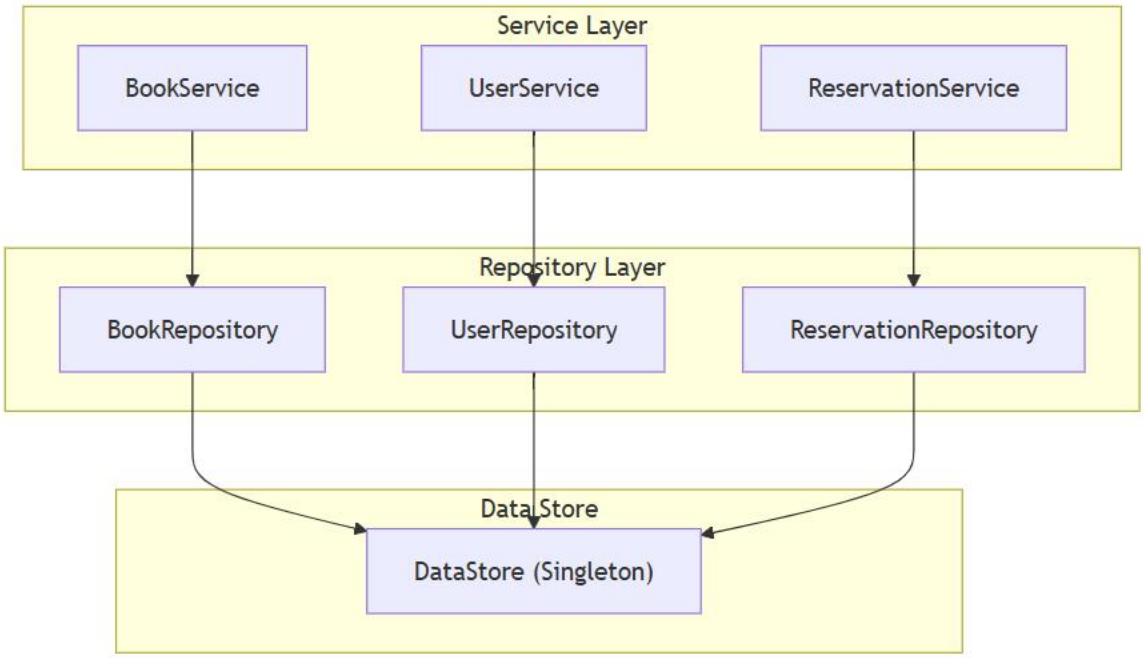
Definition

The Repository Pattern gives a set of methods/operations to retrieve stored data by adding a layer between the business logic components and data layer. This gives a system a way to store data, a way to hold complex logic, and a way to retrieve/present clean data, resulting in a system with good architecture and a clean way to separate components for unit testing.

Implementaion

The implementation of the system applies the Repository Pattern to separate the business logic from the underlying data access mechanisms. All three of the repository components (User, Book, and Reservation) serve as the boundary between the services and the data storage and provide specialized methods to read, write, and update data for domain objects

while hiding the underlying complexity of data storage (i.e. file or database) from the services.Additionally, all business logic classes (i.e. service classes) interact with the repositories and do not access the storage layer directly. This design will provide a higher level of maintainability by allowing for storage concerns to be isolated, making it easier to change or update the way data is stored (like moving from file I/O to a relational database) without impacting the higher-level logic of the system.



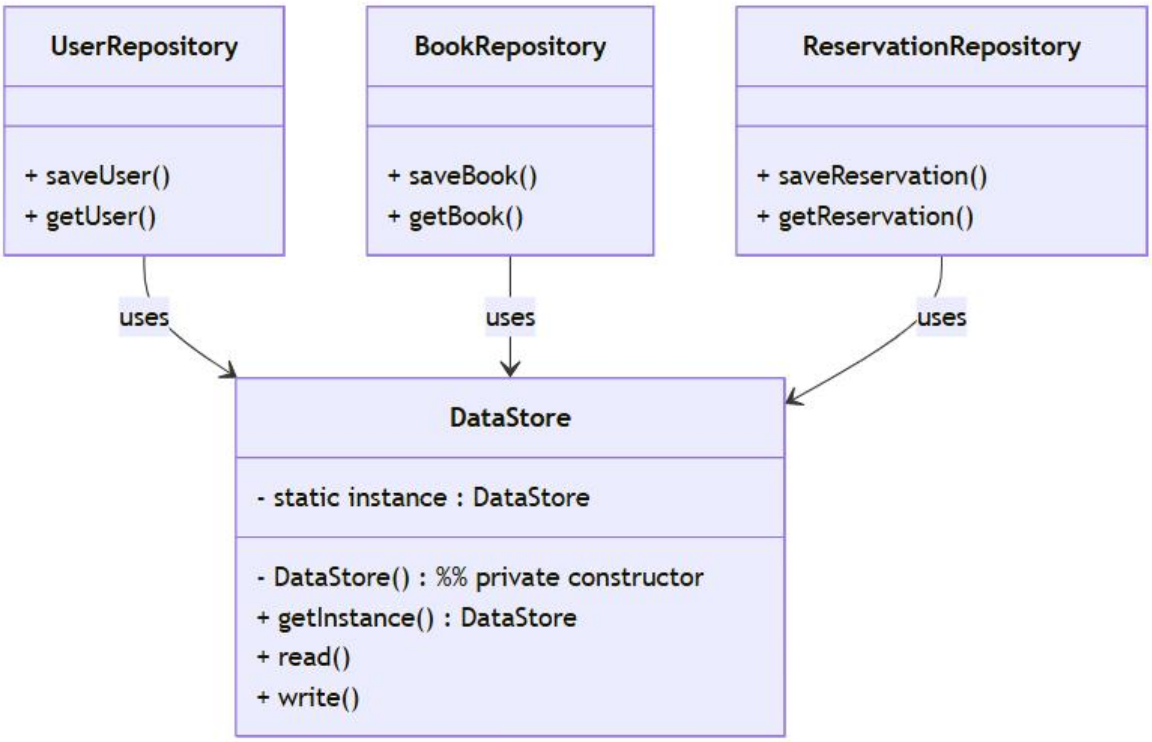
9.3 Singleton Pattern

Definition

The Singleton Pattern ensures that only one instance of a particular class exists throughout the execution of an application. It provides a global access point to this instance and prevents accidental creation of multiple conflicting objects. It is often used for managing shared resources such as configuration files, logging, and storage.

Implementation

The Datastore component is implemented using the Singleton Pattern, ensuring that only a single instance of the datastore exists throughout the execution of the system. All repository classes such as User Repository, Book Repository, and Reservation Repository access this same shared instance via a global getInstance() method. This guarantees that every read and write operation interacts with a unified storage source. By preventing the creation of multiple DataStore objects, the system achieves consistent data access, avoids conflicting file writes, and maintains centralized control over persistent storage. This design significantly reduces the risk of data corruption and ensures reliable and synchronized data management.



9.4 Factory method Pattern

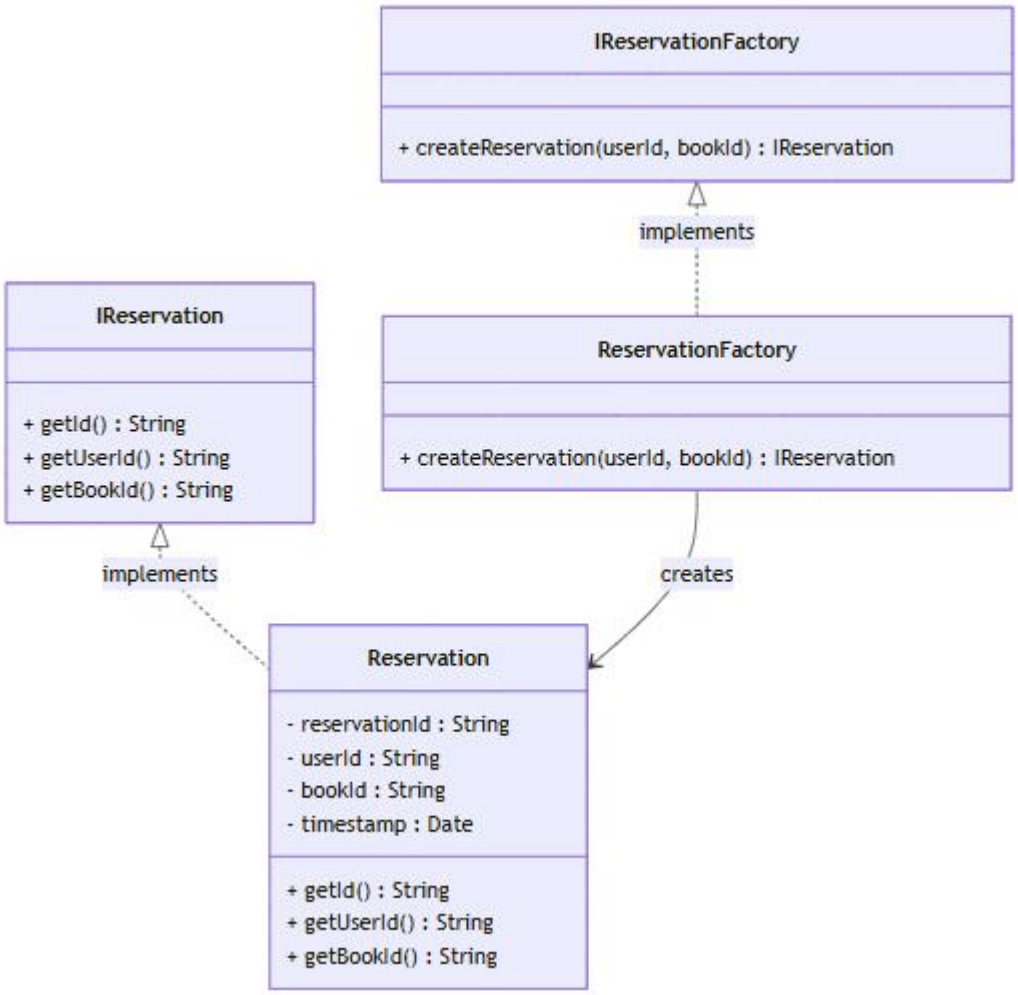
Definition

The Factory Method Pattern provides the interface of creating objects and lets the subclass (or factory class) pick which object to instantiate. It holds the complex construction logic and guarantees the same way of creation.

Implementation

The system has a ReservationFactory that helps in creating new reservations.For this system, we use the advanced version of the Factory Method Pattern to streamline and standardize the generation of reservation objects. There is the abstract creator interface (IReservationFactory) which has the factory method that is supposed to create the reservation objects. This is implemented in the concrete factory (ReservationFactory) that creates a Reservation object and has all the required fields, after which validation is done, and a unique ID is assigned, along with time stamps, and the identifiers of the book and the user are linked.

The reservation objects are handled by the product interface (IReservation), which has the base structure, and the concrete product class, Reservation, of which the class is a member. This method provides a well-defined structure, assures that the system can be extended easily in the future (e.g. adding other types of reservations), and avoids the unneeded duplication of the creation logic in the different controllers and services. This is all in accordance to the standard way of the GoF Factory Method model and optimal practices on the creation of objects in a scalable system.



9.5 Strategy Method Pattern

Definition

The strategies pattern provides algorithms in a family which can be separated, encapsulated, and replaced at runtime. Each strategy can be easily substituted in a flexible manner. Therefore modification and extension of behaviour can be done without any changes to the client code.

Implementation

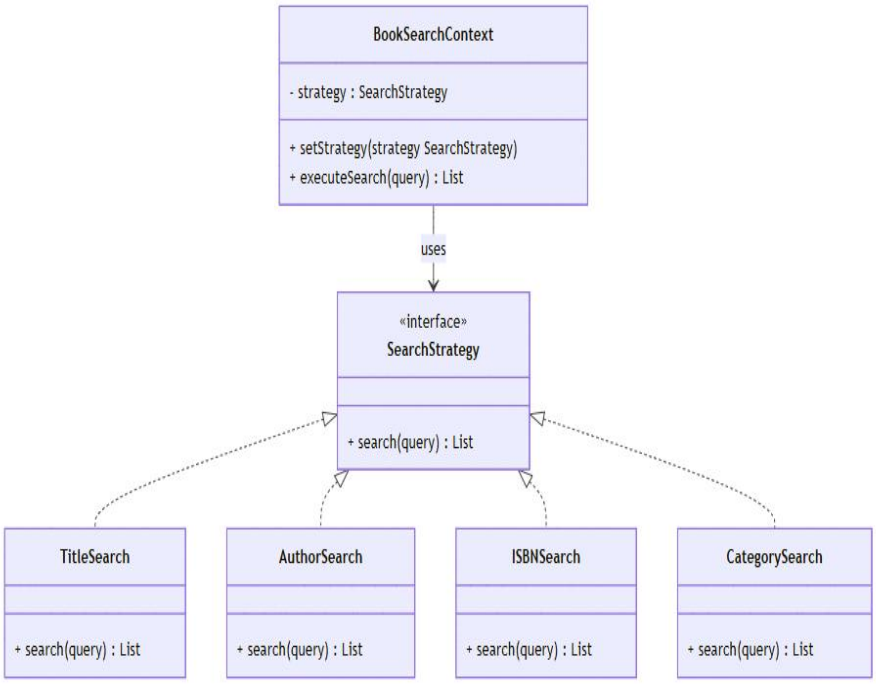
The library reservation system allows a user to search for a book in the system using different methods.

These search methods include:

- search by title
- search by author
- search by isbn

- search by category

Each search method is a different strategy class that uses the same SearchStrategy interface. Based on the user search, the controller selects the appropriate strategy at runtime.The entire search subsystem is easily extensible and modular due to this pattern.



9.6 Observer Pattern (Notifications)

Definition

The Observer Pattern is used in the Library Reservation System to automatically notify users when reservation events occur. This Pattern establishes a one-to-many association among objects such that when one object changes (Subject), all objects that depend on it (Observers) get notified of that change. The pattern helps maintain a loose coupling by decoupling the event generator from the event consumer

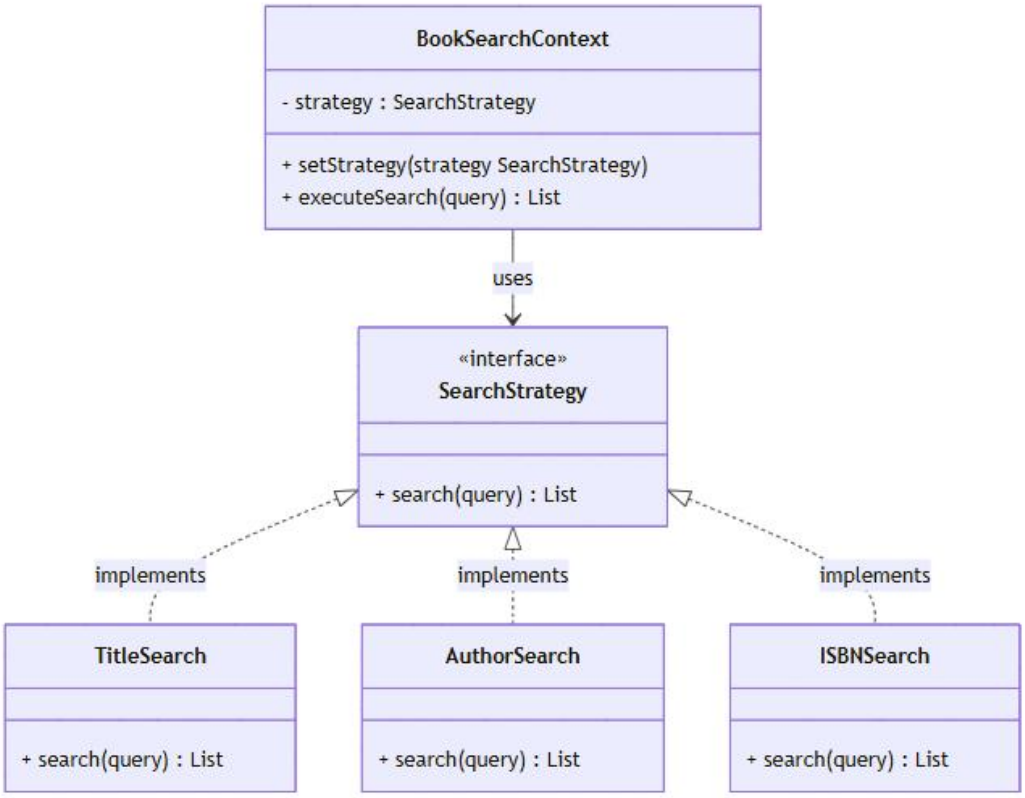
Implementation

The system employs Observer in relation to all reservation notifications.

- Subject : ReservationService
- Observers: Notification Service (email/SMS/push modules)

When a reservation is confirmed, cancelled, or expired, the subject informs all observers registered to that event. Observers then send the pertinent update to the user.

This pattern accomplishes the goal of keeping business logic decoupled from the delivery mechanisms while also providing notifications in real time.



10. Added Value

10.1 Architectural Value Through MVC Framework Design

Because of the clean separation of concerns that the Model–View–Controller (MVC) architectural pattern provides as well as the maintained scalability and sustainable framework of such architectural patterns, this method was chosen for the design of the Library Reservation System. Such systems favor this framework as there must be a separation of the user interface from the business logic and the storage of data.

The system employs:

• MODEL: These are the domain objects like a Book, User, and Reservation and are the ones who define the business logic and the state.

•VIEW: It is the user's interface that presents the information of a book, the status of a reservation, and the data from a search.

•CONTROLLER: These are the Book Controller, User Controller, and Reservation Controller, and they are there to manage the application's functionality, supervise the workflow of the users, and assign work to the services.

Such a design provided a lot of value such as accelerated system development, clarity in logic, and system flexibility to be extended or modified in individual components that were designed to be changed with system-wide stability in the overall MVC structure remaining intact. Such a structure allowed the seamless integration of additional design patterns such that more complex systems could be built.

10.2 Added Value Through Repository-Based Data Management

One of the most valuable aspects of the system is its use of the Repository Pattern, which is a standard architectural pattern that centralizes access to data that is stored. This pattern enabled the system to deal with user data, book data, and reservation data in one place and in a consistent manner.

Examples of Repositories include:

• UserRepository

• BookRepository

• ReservationRepository

These act as facilitators of communication between the controllers/services and the storage engine. By keeping storage logic behind repositories, the system is able to achieve:

✓ Quicker development

✓ More organized code

✓ Greater maintainability

✓ Ability to change databases or file types without rewriting code

This was particularly useful because it allowed future migrations to SQL, NoSQL or a cloud database to be done without affecting the application's logic on a higher level.

10.3 Centralized Data Handling Through Singleton DataStore

The DataStore operates as a Singleton. This means there is only one storage handler throughout the entire system. The DataStore reads and writes all data regarding the users, books and reservations.

The Singleton paradigm is beneficial as it:

•Avoids the complications that arise from having several instances

•Offers a certainty of having consistent and synchronized data

•Monitors the file corruption risk

• Serves as a straightforward access point

This provides the system data integrity, especially when there are multiple reservations made at the same time.

10.4 Improved Flexibility with Factory Method for Reservation Creation

The system provides additional value by adopting the Factory Method Pattern for creating reservations. Instead of scattering reservation construction logic across controllers, a dedicated ReservationFactory encapsulates all object creation steps.

The factory:

- Assigns unique IDs
- Performs validation
- Sets timestamps
- Ensures object consistency

This improves maintainability, reduces duplication, and makes the reservation subsystem extensible (e.g., premium reservations, guest reservations).

10.5 Enhanced Search Capabilities Through Strategy Pattern

The system's modular and adaptable search pattern implemented using the Strategy Pattern is one of its key assets. Searching for a book is done by:

- Book Title
- Book Author
- Book ISBN
- Book Category

Each approach is implemented as its own individual strategy class. When the strategy context is in action, it chooses which algorithm is most suitable for the moment which allows the system to:

✓ Incorporate new search parameters with ease

✓ Change algorithms as needed without making changes to the controllers

✓ Keep the code tidy, improve maintainability, and increase scalability

This is valuable to libraries with a large collection and collection that is constantly changing.

10.6 Real-Time User Engagement Through Observer-Based Notifications

To improve the user experience, the system incorporates the Observer Pattern to trigger notifications in real-time. When an automated reservation gets booked, cancelled, or expires,

real-time updates are sent to the EmailNotificationService and SMSNotificationService observers.

This results in:

• Automated notifications to users

• Decreased manual follow-up from library staff

• Enhanced user experience

• Future system extensibility to accommodate other notification types

The Observer Pattern enables enhanced responsiveness and modularity.

10.7 Improved Workflow for Library Staff and Users

All these architectural and design choices provide significant benefits to users of the library:

• Remotely search for, reserve, and track books

• Receive instant notifications about updates on reservation status

• Obtain a smooth and user-friendly experience

To the library staff:

• Less manual administrative work

• Data is maintained correctly and systematically

• Reservations are handled automatically

• System is designed to be easily modified to provide additional digital offerings in the future

10.8 Overall System-Level Added Value

Employing strong design patterns and modular architecture yields a long-lasting positive return by making the system:

- Scalable for future library growth
- Maintainable through a proper clean separation of concerns
- Extensible for new reservation rules and new search and notification types
- Consistent and reliable through central management of data
- Automated and efficient by minimizing manual processes

The convergence of these factors adds to both the operational impact of the system and the satisfaction of its users and results in a modern library management system that is easy to use and well designed.

11. Code Fragments

USER AND BOOK MANAGEMENT

```
// ---------- User & Book Management ----------

public void addUser(User user) {  3 usages  vehicle-project
    users.put(user.getId(), user);
}

public User getUser(String userId) {  2 usages  vehicle-project
    return users.get(userId);
}

public void addBook(Book book) {  4 usages  vehicle-project
    books.put(book.getId(), book);
}

public Book getBook(String bookId) {  4 usages  vehicle-project
    return books.get(bookId);
}

public Collection<Book> getAllBooks() {  1 usage  vehicle-project
    return books.values();
}

public Collection<User> getAllUsers() {  1 usage  vehicle-project
    return users.values();
}
```

LIBRARY FUNCTIONS

```
// ---------- Librarian Functions ----------

public void updateBookCopies(String bookId, int newTotal) {  1 usage   vehicle-project
    Book book = books.get(bookId);
    if (book == null) {
        throw new IllegalArgumentException("Book not found.");
    }
    if (newTotal < 0) {
        throw new IllegalArgumentException("Total copies cannot be negative.");
    }
    book.setTotalCopies(newTotal);
}

public boolean removeBook(String bookId) {  1 usage   vehicle-project
    Book book = books.get(bookId);
    if (book == null) return false;

    // Prevent removal if there are active loans
    for (Loan loan : loans) {
        if (loan.getBookId().equals(bookId) && loan.isActive()) {
            throw new IllegalStateException("Cannot remove book; there are active loans for this title.");
        }
    }
```

ADD/REMOVE RESERVATIONS

```
        // Remove reservations for this book
        reservations.removeIf( Reservation r -> r.getBookId().equals(bookId));
        books.remove(bookId);
        return true;
    }

    public void printReservationQueueForBook(String bookId) {  1 usage   vehicle-project
        List<Reservation> list = getReservationsForBook(bookId);
        if (list.isEmpty()) {
            System.out.println("No reservations for this book.");
            return;
        }
        System.out.println("Reservations for book " + bookId + " (FIFO):");
        for (Reservation r : list) {
            User u = users.get(r.getUserId());
            System.out.println("  ReservationId: " + r.getReservationId() +
                    " | User: " + (u != null ? u.getName() : r.getUserId()) +
                    " | CreatedAt: " + r.getCreatedAt());
        }
    }
}
```

12. Critique

The development of the Library Reservation System has been a valuable learning experience that significantly enhanced our understanding of software design principles but it came with its set of obstacles. Although the project began with a clear goal, the journey involved several challenges that ultimately strengthened my technical and analytical capabilities. Given the focus on system architecture, design and, particularly, UML diagrams, I had to ensure that each model I designed was a complete system on its own, and I had to work in a code-free environment to be able to reflect my system and design its behaviour. There were some diagrams that were not in sync with my requirements, and so I had to go through my use cases, class diagrams, and sequence flows several times. There was also the problem of applying a few design patterns, and in this case, it was the MVC, Repository, Strategy, Factory Method, and Observer patterns, which was difficult because every pattern had to be justified in some way by the system and not just applied for the sake of it. In the end, I was able to overcome a few barriers that made it possible for me to get a better understanding of software design principles, especially the patterns of design and their implications on the system in terms of architectural decisions or the system being performant and maintainable, along with my ability to model being significantly improved.

13.References

- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.
- Pressman, R. S., & Maxim, B. R. (2015). Software Engineering: A Practitioner's Approach. McGraw-Hill.
- Fowler, M. (2003). UML Distilled: A Brief Guide to the Standard Object Modeling Language (3rd ed.). Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- Larman, C. (2004). Applying UML and Patterns (3rd ed.). Prentice Hall.
- draw.io Documentation. (2024). Diagramming and Modeling Tool. **https://www.draw.io**